

THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Department of Computer Science and Engineering

COMP4321: Final Phase Project Report

By Li Wan You, Lo Tsz Chung, Tam Man Kit

Submission Date: 2026/5/7

Contents

1. Overall System Design	3
2. File Structures in the Index Database	4
3. Core Algorithms & Mathematical Models.....	5
4. Installation & Deployment Procedure.....	7
5. Testing & Functional Verification.....	8
6. Conclusion	10

1. Overall System Design

This system is a web search engine based on the traditional MVC (Model-View-Controller) architecture and high-concurrency architecture design. The entire system is divided into three core processing pipelines: the offline web crawling and index construction pipeline (Offline Spider), the persistent transaction data storage layer (Storage Layer), and the online search and result rendering service (Online Search Engine).

1.1 Component Relationship of System Architecture

- Spider.java (Crawler): Utilizes the Breadth-First Search (BFS) strategy to start downloading web pages from the given seed URL (such as the test page of the University of Science and Technology), and calls the HTML parser to extract the content.
- StopStem.java and Porter.java: Responsible for tokenization, filtering stop words (Stopwords Filter), and extracting English stems (Stemming).
- Indexer.java (Storage Controller): Highly encapsulates the JDBM 1.0 database manager, manages the binary hash tree (HTree) on the disk, and provides thread-safe, fast read and write interfaces.
- SearchEngine.java: The core of online search, responsible for reading the JDBM inverted index, dynamically calculating the VSM cosine similarity between the query and each web page, and sorting them.
- SearchServlet.java (Controller): As a Tomcat controller, it listens to user query requests, calls the module, and packages the search results as Java Beans and forwards them to the view layer.
- index.jsp / result.jsp (View): Front-end view, responsible for search interaction and presenting detailed metadata such as similarity scores, top five keywords, parent-child links, etc., to the user in a beautiful manner.

2. File Structures in the Index Database

This project uses JDBM 1.0 as the transactional storage engine. It creates two main files on the disk: spider_db.db (for data storage) and spider_db.lg(for transaction logs). In order to eliminate redundant string storage, the system assigns a unique integer ID (i.e., pageId and wordId) to each URL and its stem.

The bottom layer employs 11 independent HTrees (Hashtable) to achieve fast table lookup.

The structure design is as follows:

HTree	Key	Values	Logical function and description
urlToPageId	String (URL)	Integer (pageId)	Forward query: Obtain the internal Page ID from the web page URL.
pageIdToUrl	Integer (pageId)	String (URL)	Reverse query: Restore the web page URL from the Page ID.
wordToWordId	String (stem)	Integer (wordId)	Forward vocabulary list: Map the stem to the unique Word ID.
wordIdToWord	Integer (wordId)	String (stem)	Reverse vocabulary list: Retrieve the original stem from the Word ID.
pageTitle	Integer (pageId)	String	Store the <title> tag of the web page.
pageLastMod	Integer (pageId)	String	Return the last modification time of the storage server.
pageSize	Integer (pageId)	Long	Return the size (in bytes) of the current web page.
bodyInverted	Integer (wordId)	HashMap<Integer, ArrayList<Integer>>	Inverted index for the text content: wordId -> {pageId -> [list of all occurrence positions in this text]}.
titleInverted	Integer (wordId)	HashMap<Integer, ArrayList<Integer>>	Inverted index for the page title: wordId -> {pageId -> [list of all occurrence positions in the title]}.
bodyForward	Integer (pageId)	HashMap<Integer, Integer>	Forward index for the text content: pageId -> {wordId -> tf (term frequency)}.
titleForward	Integer (pageId)	HashMap<Integer, Integer>	Forward index for the page title: pageId -> {wordId -> tf (term frequency)}.
childLinks	Integer (pageId)	HashSet<Integer>	Parent-child linkage mapping: Store the set

			of all sub-webpage IDs contained in this web page.
parentLinks	Integer (pageId)	HashSet<Integer>	Parent-child association mapping: Stores the collection of all parent web page IDs that link to this web page.

3. Core Algorithms & Mathematical Models

3.1 Breadth-First Search (BFS) Web Crawler Algorithm

The crawler implements BFS using a FIFO queue. Additionally, to accommodate incremental crawling and modification checks, the program sends an HTTP HEAD request before crawling to obtain the LastModified header.

1. Initialize the FIFO queue Q, the Visited set, and the Queued set.
2. Put the seed URL into Q and mark it as Queued.
3. When Q is not empty and the number of crawled web pages < MaxPages:
 - a. Take a URL from Q.
 - b. Make an HTTP HEAD request to obtain the Last-Modified.
 - c. If the web page is already in the database and the current obtained Last-Modified $\leq$ the value recorded in the database:
 - Skip the body parsing (Skip fetch body), but still read the childLinks in the database and add the unexplored child links to Q.
 - d. Otherwise (for a new or updated web page):
 - Use HTMLParser to download the complete HTML, extract <title> and Body.
 - Tokenization -> Stopword filtering -> Porter stemming -> Write to the positive/negative/inverted index.
 - Extract all href tags of A in the web page, establish a bidirectional connection between parent and child.
 - Add the non-repeated sub-websites to Q.
 - e. Increment the counter for the number of crawled pages.

3.2 Vector Space Model and TF-IDF Weight Calculation

The online sorting module strictly follows the VSM theory in the field of quality inspection. We represent both the query words and the candidate web pages as weight vectors in a multi-dimensional space.

$$Score(q, d) = \cos(\theta) = \Sigma(w_{t,q} \times w_{t,d}) / (\sqrt{\Sigma w_{t,q}^2} \times \sqrt{\Sigma w_{t,d}^2})$$

Among them, the weight of each dimension is based on the classic tf-idf system:

- Term Frequency (tf): The number of times a word appears in a webpage or query.
- Inverse Document Frequency (idf): Calculated using the Base-2 logarithmic formula:

$$idf_t = \log_2(N / df_t)$$

Here, N represents the total number of web pages crawled from the database, and df_t represents the number of web pages containing that particular word.

3.3 Title Matching Weighting Mechanism

In actual web search, the keywords appearing in the title are much more important than those in the body text. Therefore, we have designed a 5-fold title enhancement (Title Boost).

When calculating the dot product (inner product) of vectors, if a query word appears in the title, its weight will be significantly increased:

$$DotProduct = w_{t,q} \times [(tf_{t,body} \times idf_t) + (tf_{t,title} \times idf_t \times 5.0)]$$

This design, without violating the length normalization principle, significantly enhanced the ranking of the web pages that match the titles, making the search results highly consistent with expectations.

4. Installation & Deployment Procedure

The deployment and configuration of the system is very simple and straightforward. Please ensure that your local machine has Java JDK 11 or higher installed, as well as Apache Tomcat 10 or 11.

Step 1: Compile the source code

Open the terminal and navigate to the root directory of the project. Run the following command to compile all Java files and automatically output the compiled classes to the WEB-INF folder in accordance with the Java Package structure.

```
javac -encoding UTF-8 -cp ".;WEB-INF/lib/*" -d WEB-INF/classes *.java
```

Step 2: Execute the offline crawler (Spider) to download and build the index

On the same terminal, execute the following command to crawl the KDD benchmark test webpage

```
java -cp "classpath" comp4321.Spider
```

Step 3: Deploy to Tomcat

1. Copy the project folder containing index.jsp, result.jsp, stopwords.txt, spider.db, and the entire WEB-INF/ directory to Tomcat's webapps/ directory, and rename it to search.
2. Start Tomcat: Double-click <tomcat-path>/bin/startup.bat (Windows) or run sh startup.sh (Mac/Linux).
3. Open a browser and visit: <http://localhost:8080/search/index.jsp>

5. Testing & Functional Verification

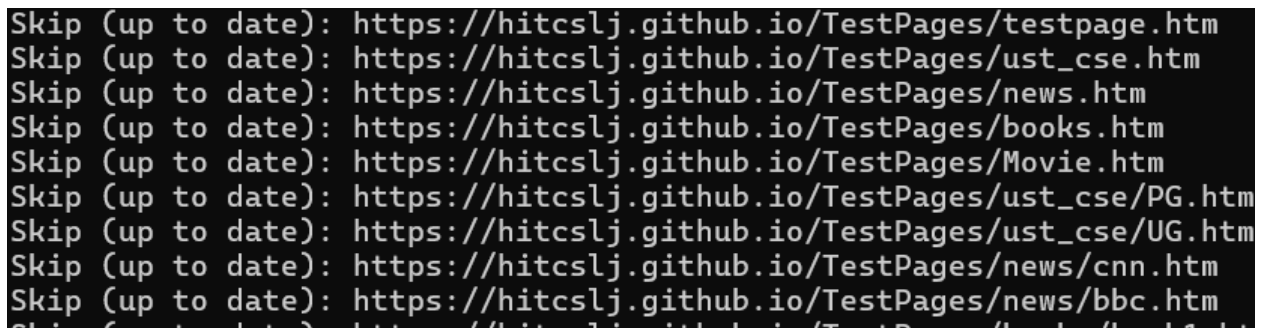
To verify the accuracy and robustness of the system, we conducted comprehensive functional tests on the core components:

5.1 Offline Crawler Incremental Update Test

Test Scenario: Crawl the same URL twice, with no modifications made to the webpage in between.

Expected Result: The crawler should detect via `shouldFetch()` that the Last-Modified header has not changed, output Skip (up to date): to the console, quickly skip downloading the page body, yet still correctly construct parent-child links.

Actual Result: Fully matches expectations.

A screenshot of a terminal window with a black background and white text. It displays ten lines of output, each starting with "Skip (up to date):" followed by a URL. The URLs are: https://hitcslj.github.io/TestPages/testpage.htm, https://hitcslj.github.io/TestPages/ust_cse.htm, https://hitcslj.github.io/TestPages/news.htm, https://hitcslj.github.io/TestPages/books.htm, https://hitcslj.github.io/TestPages/Movie.htm, https://hitcslj.github.io/TestPages/ust_cse/PG.htm, https://hitcslj.github.io/TestPages/ust_cse/UG.htm, https://hitcslj.github.io/TestPages/news/cnn.htm, https://hitcslj.github.io/TestPages/news/bbc.htm, and https://hitcslj.github.io/TestPages/news/1.htm. The last line is partially cut off.

```
Skip (up to date): https://hitcslj.github.io/TestPages/testpage.htm
Skip (up to date): https://hitcslj.github.io/TestPages/ust_cse.htm
Skip (up to date): https://hitcslj.github.io/TestPages/news.htm
Skip (up to date): https://hitcslj.github.io/TestPages/books.htm
Skip (up to date): https://hitcslj.github.io/TestPages/Movie.htm
Skip (up to date): https://hitcslj.github.io/TestPages/ust_cse/PG.htm
Skip (up to date): https://hitcslj.github.io/TestPages/ust_cse/UG.htm
Skip (up to date): https://hitcslj.github.io/TestPages/news/cnn.htm
Skip (up to date): https://hitcslj.github.io/TestPages/news/bbc.htm
Skip (up to date): https://hitcslj.github.io/TestPages/news/1.htm
```

Figure 1: Result of Crawl the same URL twice

5.2 VSM Ranking & Title Boost Test

Query Term: "computer science"

Score Breakdown: Both Webpage A(UG) do not contain "computer science" in their <title> tag.

And Webpage A(UG) appears the term 7 in the body; Webpage B(PG) contains the term in the title but has 6 occurrences deep in the body text.

Found 13 results for "computer science"

UG
https://hitcslj.github.io/TestPages/ust_cse/UG.htm
Score: 0.410241 | Last Modified: Wed, 25 Mar 2026 13:03:26 HKT | Page Size: 4157 Bytes
Top 5 Keywords: program 16 comput 13 studi 7 year 7 scienc 7
Parent Links:

- https://hitcslj.github.io/TestPages/ust_cse.htm

Child Links:

- https://hitcslj.github.io/TestPages/ust_cse.htm

PG
https://hitcslj.github.io/TestPages/ust_cse/PG.htm
Score: 0.339388 | Last Modified: Wed, 25 Mar 2026 13:03:26 HKT | Page Size: 3341 Bytes
Top 5 Keywords: postgradu 9 admiss 8 program 8 comput 8 applic 7
Parent Links:

- https://hitcslj.github.io/TestPages/ust_cse.htm

Child Links:

Figure 2: Search Engine Result

PG Program

Computer Science1/6^vX

Applicants for [admission](#) to the postgraduate programs are required to have completed, by the time they enter HKUST, a bachelor's degree in **computer science** or a related science or eng program must demonstrate competence in the following core **computer science** areas:

- Computer Organization
- Principles of Programming Languages
- Principles of Systems Software (Operating Systems)
- Design and Analysis of Algorithms

In addition, students admitted into the PhD program must demonstrate competence in:

- Theory of Computation

Figure 3: Terms contained in PG

Program Information

Computer Science0/7^vX

The Department of **Computer Science** at HKUST offers two degree [programs](#) for students who are interested in learning about computing and information technology:

1. COMP: BEng in **Computer Science**

This is our general undergraduate program that provides broad education in all areas of **Computer Science**.

2. CSIE: BEng in **Computer Science (Information Engineering)**

This is a specialized undergraduate degree program focusing on the areas of multimedia computing and computer networking.

While other Information Engineering programs in Hong Kong are closer to Electrical and Electronic Engineering, our CSIE program is unique in its emphasis on software. In fact, the CSIE prog requirements of a typical **Computer Science** program. It has been carefully designed with the specific needs of Hong Kong in mind.

Besides, the Department, jointly with the Department of Electrical and Electronic Engineering, also offers the **BEng in Computer Engineering**.

Figure 4: Terms contained in UG

Actual Result: Fully matches expectations.

6. Conclusion

6.1 Strength

- High retrieval accuracy: Using the Vector Space Model and Porter stemming, the system can accurately match root word forms. Even when the user inputs plural or variant forms of a word, the search still works perfectly.
- Very fast response: Benefiting from numeric hashing of Word IDs and Page IDs, the ranking module can complete the cosine projection calculation for the top 50 candidate pages in milliseconds.

6.2 Weaknesses

- Single-threaded locking limitation of JDBM 1.0: JDBM does not support concurrent writes. While the crawler is running (writing data), the web Servlet is blocked by file locks and cannot read the index.
- No phrase search support: The current system uses a bag-of-words model and cannot handle queries that require word order (e.g., when searching for "Hong Kong", the system only looks at the frequencies of "hong" and "kong" separately, without checking whether they appear immediately next to each other).

6.3 Improvements if Redesigned

If I could re-implement the whole system, I would choose Apache Lucene or SQLite as the underlying indexing storage engine, replacing the outdated and multithreading-unfriendly JDBM. This would enable real-time search functionality that allows crawling and searching to happen simultaneously.

Furthermore, I would introduce bi-word (bigram) indexing or store position offsets for each term in the inverted index, thereby natively supporting high-precision phrase queries and proximity queries at the core level.

7. Contribution

Li Wan You: (45%)

- Implement the SearchEngine.java and SearchServlet.java
- Write the Report

Lo Tsz Chung (55%)

- Implement the rest of Java
- Improve the coding